Documentation Analyse Energétique

Projet Génie Logiciel

Groupe 1, gl06

ZIVKOVIC Lucas, SULLET Benjamin, ZOUGGARI Samy,
TRAORÉ Fiè Victor, HURSTEL Mathis

Janvier 2025

Grenoble INP - Ensimag
Génie Logiciel 2024-2025

Table des matières

1 Efficience du Code Produit **2**
1.1 Évaluation du temps de quelques instructions 2
1.2 Résultats de performance avec IMA -s 2
1.3 Interprétation et analyse 2
1.4 Coût moyen des instructions 3

2 Efficience du Procédé de Fabrication **3**
2.1 Coût énergétique des tests 3
2.2 Bilan énergétique 4

3 Annexe : Rappel coût des instructions IMA **4**

1 Efficience du Code Produit

Dans cette partie de l'analyse, nous nous baserons sur le nombre de cycles des instructions IMA pour apprécier la qualité du code produit.

1.1 Évaluation du temps de quelques instructions

Programme DECA :

```
{
    int x = 1;
    println(x);
}
```

Nous pouvons générer plusieurs types de code pour ce programme. En considérant que 1 est stocké à l'adresse 1(GB), nous pouvons avoir les deux codes assembleur suivants pour l'affichage de x :

Programme IMA 1 :

```
LOAD 1(GB), R1
WINT
WNL
HALT
```

Programme IMA 2 :

```
LOAD #0, R1
LEA 1(GB), R1
LEA 0(R1, R0), R1
LOAD 0(R1), R1
WINT
WNL
HALT
```

1.2 Résultats de performance avec IMA -s

- Pour le premier code assembleur :
  - Nombre d'instructions : 6
  - Temps d'exécution : 47
- Pour le second code assembleur :
  - Nombre d'instructions : 9
  - Temps d'exécution : 60

1.3 Interprétation et analyse

On remarque que le deuxième mode d'adressage consomme environ 30% plus que le premier mode. Nous avons donc opté pour le premier mode d'adressage qui est, de plus, assez intuitif.

Il est important de mentionner cela car nous avions envisagé de généraliser l'accès à la pile par ce mode d'adressage indirect pour éviter de redemander l'adresse d'un objet

dans notre code Java. Cependant, cela n'est pas performant car un objet peut être dans le tas, et nous serions amenés de toute façon à recalculer encore l'adresse indirecte.

Pour notre extension TAB, pour accéder aux éléments d'un tableau, nous n'avons pas eu d'autre choix que ce mode d'adressage. Cela coûte donc un peu plus cher d'accéder à l'élément 0 d'un tableau par rapport à un simple entier dans une variable.

Note : L'instruction DEL permet de libérer du tas après en avoir alloué avec NEW. Nous n'utilisons pas cette instruction coûteuse (16 cycles) car il n'est pas question d'instruction de "free" ou "libération" dans la syntaxe de DECA. Nous aurions pu le gérer en interne, mais son utilité ne nous a pas paru essentielle.

1.4 Coût moyen des instructions

Instructions	**Coût moyen (cycles/temps)**
Déclarations (int, bool, new, ...)	27+3+1+3/11+1+11+66
If elsif *n else	5+4*n/16
a || b	23/58
a && b	8/30
Classe avec n fields et m méthodes	3*n + 4*m / 59+m*10
a b, a > b, a b, a < b	6/24
while	5/22

Notes importantes :

- Pour une instruction new, il faut prendre en compte les instructions pour l'initialisation qui dépendront de la taille de la classe en termes de champs.
- Pour les if/else : ce coût représente juste l'instruction, il faut encore ajouter le coût des conditions effectives et des instructions dans les blocs.
- Pour les opérations élémentaires (+, -, *, /), se référer au coût de IMA.
- Pour les classes, ce coût représente juste celle de la structure, et il faut en plus tenir compte des coûts des instructions et des déclarations à l'intérieur.

2 Efficience du Procédé de Fabrication

2.1 Coût énergétique des tests

- Coût (test_lex + test_context + test_synt) : environ 600ms par test
- Coût (decac) : environ 200ms par test
- Coût d'exécution de nos tests :
  - Nombre de cycles IMA : 2490
  - Temps : 20128

Nous estimons avoir lancé nos tests environ 200 fois par personne et par test, soit un total d'environ 1000 lancements, représentant environ 600s de coût direct pour les tests.

Pour la commande decac, nous estimons l'avoir lancée environ 5000 fois, soit :

— Total cycles : 2490 * 5000 = 12450000

— Total temps : 20128 * 5000

2.2 Bilan énergétique

En considérant les temps de fonctionnement de nos ordinateurs pour le projet (10H/personne/jour), et une puissance de 0,1kW pour nos machines, la consommation est de :

$$10 * 5 * 30 * 0,1 = 150 \text{ kWh}$$

Au final, le bilan énergétique de notre projet est raisonnable. En privilégiant l'adressage direct par rapport à l'adressage indirect, nous pensons avoir amélioré légèrement l'efficacité énergétique. De plus, nous n'avons pas pris en compte les centaines de compilations effectuées via la commande mvn compile, car cela est intégré dans le fonctionnement global de l'ordinateur.

3 Annexe : Rappel coût des instructions IMA

Instruction	**Cycles**	**Instruction**	**Cycles**
LOAD	2	DIV	40
STORE	2	INT	4
LEA	0	BRA	5
PEA	4	Bcc	5/4
PUSH	4	BSR	9
POP	2	RTS	8
NEW	16	DEL	16
ADD/SUB	2	RINT	16
SHL/SHR	2	RFLOAT	16
OPP	2	WINT/WFLOAT	16
MUL	20	RUTF8/WUTF8	16
CMP	2	WSTR	16
QUO/REM	40	WNL	14
FLOAT	4	ADDSP/SUBSP	4
Scc	3/2	TSTO	4
ERROR	1	HALT	1
SETROUND	20	SCLK	2
FMA	21	CLK	16